

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computación
Diseño de Software

Tercer Proyecto: Servidor nivel producción en Azure para un juego por turnos

ASHENFALL, servidor autoritativo para un duelo 1v1 por turnos de proyectiles

Estudiante: Kevin Miranda Méndez
Carné 2024120764

Profesor: Jonathan Solis Parajeles

Juego en línea:
https://ashenfall-server-prod.icytree-9cfd11e3.canadacentral.azurecontainerapps.io
Repositorio: https://github.com/kevinmirandaa/ashenfall-game
Versión evaluada: etiqueta v1.0.0

II Semestre 2026

1. Introducción

Este documento presenta el diseño, la construcción y el despliegue del tercer proyecto del curso Diseño de Software: un servidor de nivel producción en Azure para un juego multijugador por turnos. El enunciado pide que cada equipo invente su propio juego y que lo que se evalúe sea el servidor: qué tan bien está construido, qué tan seguro y eficiente es y cuánto sigue los estándares de la industria. El juego es el pretexto, no el producto.

Construí **ASHENFALL**, un duelo 1v1 por turnos de proyectiles, de fantasía oscura y pixel art, que se juega en el navegador. El servidor es autoritativo: valida cada jugada, simula el tiro, decide el daño y es la única fuente de verdad. Cada partida es un registro de eventos en PostgreSQL con número de secuencia, se puede reproducir jugada por jugada y sobrevive a reinicios y a publicaciones de versiones nuevas. El servidor corre en Azure Container Apps con dos réplicas, Redis, PostgreSQL, Key Vault e identidad administrada, y se despliega con Bicep y GitHub Actions.

El documento explica el juego y cómo cumple las restricciones del enunciado, los requisitos funcionales y técnicos del servidor, la arquitectura y los patrones usados, la seguridad, el rendimiento medido, la infraestructura en Azure, las pruebas, cómo se juega y cómo se instala, y al final una revisión del cumplimiento del enunciado con sus limitaciones. Todas las cifras salen de ejecuciones reales entre el 5 y el 7 de noviembre de 2026; donde algo no se pudo medir, se dice.

1.1 Enlaces

Qué	Dónde
Juego en línea (Azure, disponible durante la revisión)	https://ashenfall-server-prod.icytree-9cfd11e3.canadacentral.azurecontainerapps.io
Documentación de la API REST (OpenAPI 3.1)	https://ashenfall-server-prod.icytree-9cfd11e3.canadacentral.azurecontainerapps.io/docs
Repositorio en GitHub (público)	https://github.com/kevinmirandaa/ashenfall-game
Versión evaluada	Etiqueta **v1.0.0** del repositorio
Tablero de trabajo (Jira, proyecto SCRUM)	kevinmirandaa04.atlassian.net, proyecto SCRUM
Documentación detallada en el repositorio	Carpeta `docs/`, con índice en `docs/README.md`

2. Juego escogido y objetivo

Historia. Hace siglos, el mundo quedó dividido por un acontecimiento conocido como La Caída de las Cenizas. Las antiguas tierras dejaron de ser seguras y criaturas de las profundidades empezaron a aparecer en bosques, montañas y pueblos abandonados. En medio de ese mundo existe Ashenfall, un pequeño asentamiento comercial donde todavía se reúnen guerreros de todas las regiones, se reparan armas, se intercambian reliquias y se aceptan desafíos.

El duelo. Dos mercenarios se enfrentan en una arena y se turnan para disparar proyectiles con un gesto de tirachinas: arrastran hacia atrás para dar potencia y ángulo, y sueltan. Cada uno empieza con 100 puntos de vida, el turno dura 30 segundos y la partida tiene como máximo 12 rondas. Gana quien deja al rival sin vida; si se acaban las rondas gana quien conserve más vida, y con la misma vida hay empate. Nadie se mueve por su cuenta: solo se desplaza al ser impactado, retrocediendo según el daño recibido. No hay viento.

Modos. Buscar partida (cola rápida contra otra persona), partida privada con código de invitación y modo historia contra diez monstruos que mueve el servidor. Ganar un duelo da 100 Marcas de Ceniza; con ellas y con el nivel se compran nueve personajes más en el Arsenal, pues se empieza con Edrik. Hay siete arenas (aldea en ruinas, cementerio, desierto, pinar con niebla, puente helado, fortaleza carmesí y ruinas brumosas) y en cada partida el servidor sortea el terreno (llano, plataformas con gradas o colina) y los obstáculos.

Objetivo de diseño. Que el juego sirva para ejercitar los problemas reales de un servidor multijugador: muchas partidas simultáneas e independientes, orden de las jugadas, mensajes duplicados, desconexiones, varias réplicas atendiendo la misma partida, información que no debe filtrarse al rival y publicaciones sin cortar partidas.

2.1 Cumplimiento de las restricciones del juego

Restricción del enunciado	Cómo la cumple ASHENFALL
Por turnos: la partida sabe a quién le toca	El estado guarda el jugador en turno y su plazo; solo ese jugador puede disparar y el temporizador de 30 s lo controla el servidor.
Multijugador: mínimo 2 jugadores humanos	Cada partida enfrenta a 2 humanos (cola rápida o sala privada). El modo historia es un extra contra un bot del servidor.
Original, no una recreación 1 a 1	Reglas propias: duelo de proyectiles con física determinista, retroceso por impacto, terreno y obstáculos sorteados, personajes comprables con nivel y dinero.
Reglas verificables	El dominio decide sin ambigüedad si una jugada es válida y cuándo termina la partida (vida, rondas, abandono); está cubierto por pruebas unitarias.
Información oculta (recomendada)	La posición del rival y la trayectoria son públicas por decisión de diseño (la cámara las muestra a ambos). Aun así hay vista por jugador: el rival nunca recibe el ángulo, la potencia ni el identificador de la jugada, y una prueba lo verifica en mensajes, REST y logs.

3. Requisitos funcionales del servidor

La API REST tiene 30 rutas y 33 operaciones documentadas en OpenAPI 3.1; el tiempo real usa un solo canal WebSocket documentado en AsyncAPI 3.0. La siguiente tabla relaciona cada capacidad pedida con su implementación.

Capacidad	Qué hace	Implementación
Usuarios	Registro, inicio y cierre de sesión, renovación del token y perfil.	POST /v1/auth/register, login, refresh y logout; GET /v1/me. JWT de 15 minutos con refresh token rotativo; contraseñas con Argon2id.
Salas	Crear partida pública o privada con código, listar, unirse, salir y marcar «listo».	POST y GET /v1/rooms, join, join-by-code, leave y ready. Inicia solo con 2 jugadores listos y la capacidad se respeta aunque varios se unan a la vez (prueba de concurrencia).
Emparejamiento	Cola rápida que agrupa jugadores en partidas nuevas.	POST, GET y DELETE /v1/matchmaking. Un jugador nunca queda en dos partidas; espera máxima de 60 s, configurable con MATCHMAKING_MAX_WAIT_MS.
Turnos	Enviar jugada, recibir el nuevo estado y saber a quién le toca.	Mensaje shoot por WebSocket. Solo juega quien tiene el turno; la jugada inválida recibe un error tipado (NOT_YOUR_TURN, STALE_VERSION…); el temporizador de 30 s tiene como acción por defecto perder el turno.
Conexión	Detectar caídas, reconectar y recuperar el estado.	Heartbeat ping/pong, ticket de un solo uso, snapshot al reconectar y periodo de gracia de 60 s configurable (RECONNECT_GRACE_MS); al vencer, abandono.
Fin de partida	Declarar ganador o empate y registrar el resultado.	El resultado se guarda una vez y no cambia (tabla match_results); GET /v1/matches/{id}/replay reproduce la partida jugada por jugada.
Historial	Partidas jugadas y resultados por usuario.	GET /v1/me/matches, paginado; el replay y el detalle solo los ve un participante (404 idéntico al de un recurso inexistente).
Administración	Ver partidas activas y conexiones, terminar una partida, suspender un usuario.	GET /v1/admin/matches, POST terminate, suspend y unsuspend, GET /v1/admin/audit. Solo el rol administrador; cada acción queda auditada con autor, motivo y fecha.

3.1 Reglas del servidor

Regla del enunciado	Cómo se cumple
Servidor autoritativo	El cliente envía ángulo y potencia, nunca resultados. El dominio simula el tiro y decide el daño.
Orden por partida y número de secuencia	Las jugadas de una partida se procesan una a una (fila por partida en cada réplica) y cada evento lleva un seq. Entre réplicas, la clave primaria (match_id, seq) deja confirmar a una sola; la otra relee el log y decide de nuevo. Una jugada sobre una versión vieja se rechaza (STALE_VERSION).
Idempotencia	Cada jugada lleva un actionId generado por el cliente. Si se reenvía, el servidor responde con un ack marcado duplicate y no la aplica dos veces.
Vista por jugador	Cada evento se proyecta a la vista de su destinatario. Ningún mensaje, endpoint ni log filtra el ángulo, la potencia ni el actionId del rival.
Durabilidad	El estado se reconstruye siempre desde PostgreSQL; ninguna jugada confirmada vive solo en memoria. Si una réplica se reinicia o se publica una versión nueva, las partidas continúan.
Aislamiento	Un fallo en una partida no bloquea la fila de otra ni tumba el proceso: errores contenidos por mensaje y por partida.
Limpieza	Los plazos de cada partida están en PostgreSQL y todas las réplicas barren los vencidos; las partidas inactivas terminan, las salas sin iniciar se cierran a la hora y la cola filtra por expiración.
Reloj y azar del servidor	Temporizadores y sorteos (terreno, obstáculos) se resuelven en el servidor con un generador criptográficamente seguro que se inyecta al dominio.

4. Arquitectura y patrones de diseño

4.1 Tecnologías

Elemento	Tecnología
Lenguaje	TypeScript en modo estricto (strict, sin any) sobre Node.js 22
Servidor	Fastify 5, WebSocket con la librería ws, validación con esquemas TypeBox
Datos	PostgreSQL 16 (eventos y datos duraderos) y Redis 7 (difusión, presencia y límites de tasa)
Seguridad	Argon2id, JWT firmados, refresh token rotativo, ticket de un solo uso para el WebSocket
Contratos	OpenAPI 3.1 (en /docs), AsyncAPI 3.0, errores RFC 9457 Problem Details
Cliente de prueba	Phaser 3 con Vite, servido por el propio servidor
Pruebas y calidad	Vitest, ESLint con typescript-eslint estricto, tsc, CodeQL, gitleaks, Trivy y Dependabot
Infraestructura	Azure Container Apps, PostgreSQL Flexible Server, Key Vault, Container Registry, Application Insights; Bicep y GitHub Actions

4.2 Capas

El motor de reglas vive en un módulo de dominio puro (packages/domain) sin dependencias de red, base de datos ni framework. El servidor conoce al dominio y el dominio no conoce a nadie; ESLint hace cumplir esa regla y no deja importar red, base de datos, reloj ni azar globales dentro del dominio. Por eso las reglas, la física y la vista por jugador se prueban sin ninguna dependencia externa.

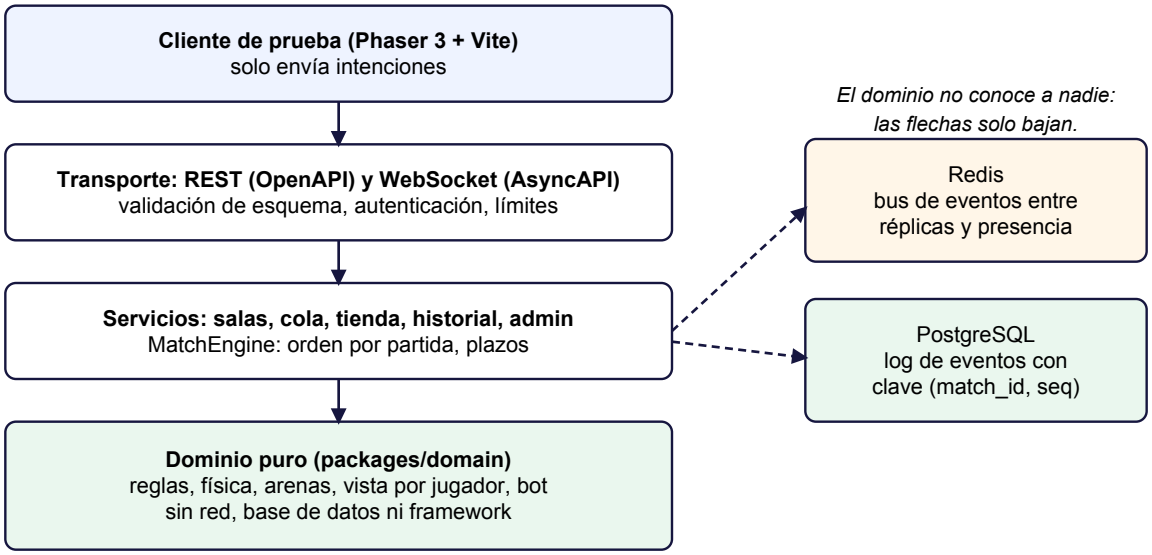


Figura 1. Capas del código: las dependencias solo bajan hacia el dominio puro.

4.3 Topología en Azure

Los jugadores se conectan por HTTPS y WSS a cualquier réplica del servidor. El estado caliente y la difusión de eventos entre réplicas pasan por Redis, las jugadas confirmadas se guardan en PostgreSQL y los secretos se leen de Key Vault con identidad administrada. PostgreSQL no tiene acceso público: vive en una subred delegada de una red virtual; Redis solo tiene ingress interno.

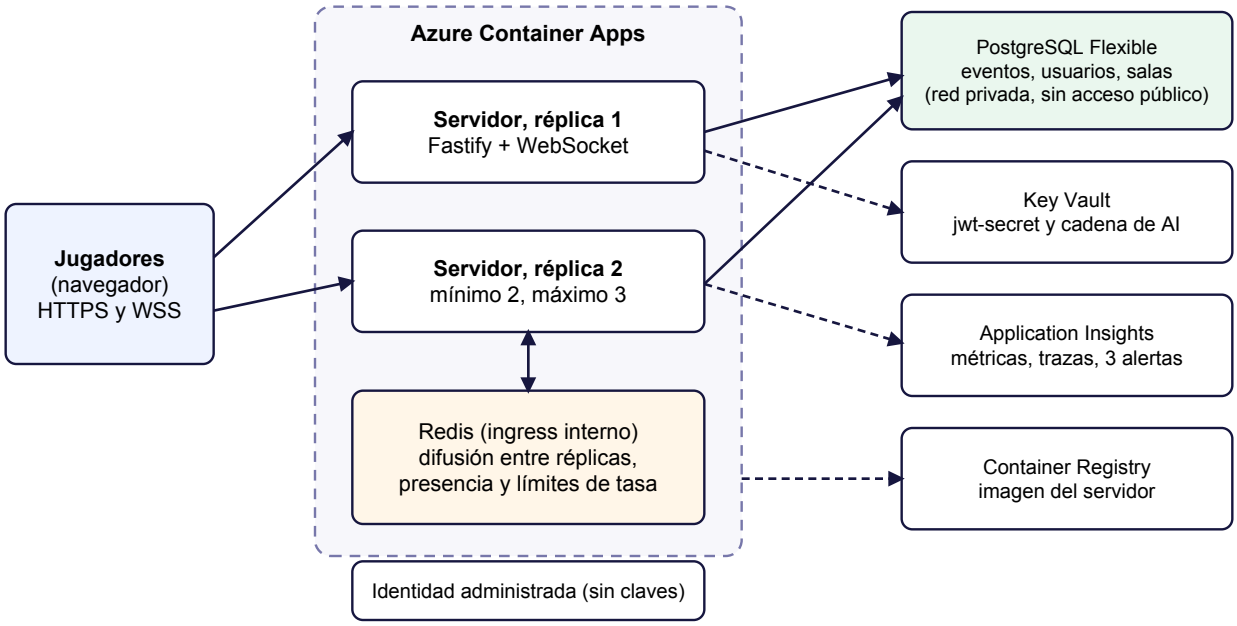


Figura 2. Topología de producción en Azure (región canadacentral).

4.4 Patrones usados

Patrón	Dónde y para qué
Event sourcing	Cada partida es un log de eventos ordenados por seq; el estado es el resultado de reproducirlo (decide, evolve y replay en el dominio). Da durabilidad, reproducción jugada por jugada y recuperación tras reinicios.
Comando y decisión pura	Un comando (disparar, agotar tiempo, desconectar) entra al dominio, que devuelve una decisión: aplicada con sus eventos, duplicada o rechazada con un código tipado.
Cola por partida y control optimista	Fila por partida en cada réplica y clave (match_id, seq) entre réplicas, con hasta 5 reintentos al perder un conflicto.
Publicación y suscripción	Bus de eventos sobre Redis: cada réplica entrega a sus conexiones locales los eventos ya confirmados. Si se pierde un mensaje, el cliente detecta el hueco en seq y pide el estado con sync.
Vista por jugador (proyección)	toPlayerView y toPlayerEvent proyectan estado y eventos a lo que cada jugador puede ver.
Inyección de dependencias	El azar (RandomSource), el reloj y la base de datos entran al dominio y a los servicios desde afuera, lo que permite probarlos con simulaciones.
Barrido de plazos sin líder	Todas las réplicas barren los plazos vencidos; si dos coinciden, el dominio rechaza el comando repetido y la clave de secuencia deja confirmar uno solo.

4.5 Flujo de una jugada

- El cliente envía `shoot` con actionId, expectedVersion, arma, ángulo y potencia.
- La pasarela valida el esquema, el tamaño y la tasa de mensajes antes de que lleguen al dominio; un mensaje inválido se rechaza sin cerrar la partida.
- El motor encola la jugada en la fila de la partida, carga el log desde PostgreSQL y reproduce el estado.
- El dominio decide: ¿es su turno?, ¿la versión es vigente?, ¿el actionId ya se aplicó?
- Si se aplica, se insertan los eventos con clave (match_id, seq); si otra réplica ganó ese seq, se relee y se decide de nuevo.
- Los eventos confirmados se publican por Redis; cada réplica los entrega con la vista de cada jugador y el tirador recibe su ack.

4.6 Ciclo de vida de una partida

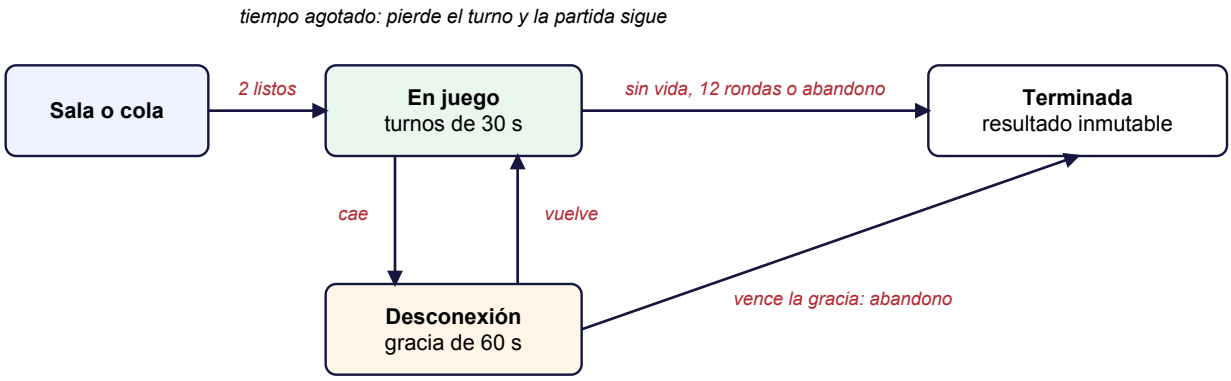


Figura 3. Ciclo de vida de una partida.

Los plazos de cada partida están en PostgreSQL (matches.next_deadline_ms) y todas las réplicas barren los vencidos, así que un reinicio no pierde temporizadores. Si un turno se agota, el jugador pierde el turno sin daño; dos turnos agotados seguidos cuentan como abandono. Al desconectarse, el jugador tiene 60 segundos para volver y recibe el estado actual; si no vuelve, pierde por abandono.

5. Contratos y protocolo

REST. Usuarios, salas, emparejamiento, historial, tienda y administración están en OpenAPI 3.1, publicada en /docs. Los errores siguen RFC 9457 (Problem Details) con un código propio, por ejemplo VALIDATION_FAILED. Los límites de tasa responden 429 con la cabecera Retry-After.

WebSocket. Un solo canal con mensajes versionados (campo v) documentados en AsyncAPI 3.0: del cliente, shoot y sync; del servidor, welcome, snapshot, event, ack y error. Cada mensaje tiene esquema, ejemplos y errores posibles. La conexión se autentica en el handshake con un ticket de un solo uso de 30 segundos; sin ticket válido se cierra.

Código de cierre	Significado
1001	Cierre ordenado del servidor (publicación o reinicio): el cliente reconecta.
4003	Usuario suspendido por un administrador.
4008	Demasiadas conexiones para ese usuario.
4009	Se perdió el heartbeat.
4029	Demasiados mensajes por segundo: límite de tasa por conexión.

Los contratos se generan desde el código y una prueba falla si dejan de coincidir con él: contracts/openapi.json y contracts/asyncapi.json están sincronizados con las rutas y los mensajes reales.

6. Seguridad

El modelo de amenazas completo (activos, fronteras de confianza, amenazas con su defensa y su prueba, y riesgos aceptados) está en docs/threat-model.md. Resumen de las defensas principales:

Amenaza	Defensa	Prueba
Fuerza bruta de contraseñas	Límite por IP y por cuenta; Argon2id; verificación señuelo si el correo no existe	rate-limit.test.ts, auth.test.ts
Robo de refresh token	Rotación; reutilizar uno revoca la familia; solo se guarda el hash	auth.test.ts
JWT falsificado	Algoritmo fijo; emisor y audiencia verificados	auth.test.ts
Secuestro del WebSocket desde otra web	Ticket de un solo uso de 30 s y orígenes restringidos	gateway.test.ts
Enviar daño o resultados	El esquema rechaza campos extra; el servidor calcula todo	gateway.test.ts
Jugar fuera de turno o sobre estado viejo	El dominio valida turno y versión	match.test.ts, gateway.test.ts
Reenviar una jugada	actionId idempotente y clave (match_id, seq)	gateway.test.ts, resilience.test.ts
Ver datos del rival o partidas ajenas	Vista por jugador; 404 idéntico al de un recurso inexistente	view.test.ts, history.test.ts
Duplicar monedas o fijar un precio	Precio del servidor, billetera bloqueada y claves únicas	shop.test.ts, event-store.test.ts
Inundar la API o el WebSocket	Límites por usuario e IP (429 con Retry-After), tamaño máximo de mensaje, tasa por conexión y conexiones por usuario	rate-limit.test.ts, gateway.test.ts
Pico de logins que agota CPU o memoria	Argon2id acotado con fila y respuesta 503	password-hasher.test.ts
Usar funciones de administrador	Rol leído de la base en cada petición; ninguna ruta lo concede; auditoría	admin.test.ts

Amenaza	Defensa	Prueba
Secreto en el repositorio o en logs	gitleaks en CI; sin valores por defecto; logs sin cabeceras ni cuerpos	CI, hardening.test.ts
Imagen o dependencias comprometidas	Usuario no root; Trivy, CodeQL y Dependabot en CI	CI

6.1 Secretos, identidad y transporte

- El secreto de los JWT vive en Key Vault y la aplicación lo lee con su identidad administrada; no hay ningún secreto en el repositorio ni en texto plano en la configuración.
- PostgreSQL se accede con identidad de Microsoft Entra (contraseña desactivada); la cadena de conexión no trae clave.
- El despliegue desde GitHub usa OIDC federado, sin secretos guardados en el repositorio.
- Solo HTTPS y WSS: `http://` redirige con 301, hay HSTS, CSP y nosniff; CORS y los orígenes del WebSocket están restringidos (un origen ajeno recibe 403).

6.2 Riesgos aceptados

- La posición del rival es pública por decisión de diseño; un cliente modificado solo obtiene lo que el jugador ya ve, aunque podría calcular el tiro perfecto.
- Agotar el límite de login por cuenta bloquea a su dueño un minuto; se prefiere a permitir fuerza bruta distribuida.
- Con Redis caído, los límites de tasa por plugin dejan de aplicarse para no tumbar el servicio y no se pueden emitir tickets nuevos.
- Una réplica que cae de golpe no registra desconexiones; sus partidas terminan por los plazos de turno.

7. Rendimiento y optimización

La prueba de carga es reproducible: el bot de apps/bot juega partidas reales por HTTP y WebSocket y mide el tiempo desde que se envía una jugada hasta que ambos jugadores reciben el resultado. Las metas del enunciado son p95 menor a 200 ms, ninguna jugada perdida o duplicada y error menor a 1 %. Las pruebas locales se hicieron con una réplica limitada a 1 CPU y 1 GB, con PostgreSQL y Redis sin límite y el bot en la misma máquina (5 de noviembre de 2026).

7.1 Ritmo normal: un disparo por segundo y partida

Sin jugadas perdidas ni duplicadas y sin errores en ningún escalón.

Partidas	Conexiones	Jugadas	Jugadas/s	p50	p95	p99	CPU réplica	Memoria máx.
25	50	300	23	9 ms	15 ms	16 ms	11 %	53 MB
50	100	600	46	7 ms	13 ms	17 ms	13 %	55 MB

Partidas	Conexiones	Jugadas	Jugadas/s	p50	p95	p99	CPU réplica	Memoria máx.
100	200	1 200	92	4 ms	14 ms	17 ms	13 %	56 MB
200	400	2 400	184	3 ms	9 ms	24 ms	15 %	61 MB
400	800	4 800	369	2 ms	5 ms	7 ms	21 %	66 MB
800	1 600	9 600	737	2 ms	7 ms	17 ms	33 %	75 MB

Entre 400 y 800 partidas, el consumo es de unos 0,03 puntos de CPU y 0,02 MB por partida activa (la memoria incluye lo que el proceso ya tenía reservado, así que es una aproximación).

7.2 Punto de quiebre

Con diez disparos por segundo y partida, para saturar sin crear miles de cuentas. Ninguna jugada se perdió ni duplicó, tampoco al degradarse.

Partidas	Jugadas/s	p50	p95	p99	CPU réplica	CPU PostgreSQL	Meta p95
200	1 692	7 ms	43 ms	52 ms	51 %	46 %	Cumple
400	1 781	119 ms	198 ms	225 ms	66 %	95 %	En el límite
800	1 845	327 ms	428 ms	453 ms	98 %	80 %	No cumple

Una réplica deja de cumplir la meta alrededor de **1 800 jugadas por segundo**. A un disparo por segundo y partida se midió sin degradación hasta 800 partidas; extrapolando, el tope rondaría 1 700 partidas, pero esa cifra no se midió.

7.3 Optimizaciones medidas

Medida	Antes	Después
p95 de las jugadas con 100 clientes enviando logins fallidos sin parar (Argon2id acotado a una contraseña a la vez)	202 ms	109 ms
Preparar 100 partidas con sus 200 cuentas (Argon2id con parámetros mínimos de OWASP)	57 s	5 s
Idas y vueltas a PostgreSQL por jugada	7	5
Jugadas por segundo en saturación (400 y 800 partidas, dos ejecuciones)	1 593 a 1 647	1 754 a 1 839
Listado de salas públicas (50 salas)	101 consultas, p95 7,5 ms	2 consultas, p95 1,1 ms
Descarga de un jugador nuevo hasta empezar una partida	105 peticiones, 3,84 MB	47 peticiones, 1,29 MB
Imagen Docker (disco)	565 MB	387 MB
Consultas a PostgreSQL en reposo	4,42 transacciones/s	2,76 transacciones/s

Coste aceptado de la primera optimización: una sola réplica procesa unos 20 logins por segundo en un pico, así que en un ataque algunos usuarios reciben 503 y deben reintentar; se prefiere a

poner en riesgo las partidas en curso. El detalle de cada cambio está en docs/optimizacion.md.

7.4 Mensajes compactos

El WebSocket usa JSON con los campos mínimos y trayectorias en enteros muestreados: unos 302 bytes por mensaje de media; el estado completo solo viaja al conectar o al pedir sync, y el resto del tiempo se envían solo los eventos nuevos.

7.5 Mediciones en Azure (7 de noviembre de 2026)

Contra la URL pública de producción, con el bot en un portátil en Costa Rica, por lo que las latencias de extremo a extremo incluyen el viaje por internet hasta Canadá.

Prueba	Partidas en paralelo	Duración	Jugadas co nfirmadas	Partidas contrastadas	Perdidas / duplicadas / errores
Control, 1 réplica	6	11,2 min	473	18	0 / 0 / 0
Durante la publicación de la revisión con 2 réplicas	6	6,9 min	288	12	0 / 0 / 0
Sobre las 2 réplicas nuevas	8	7,1 min	518	16	0 / 0 / 0

- Cada partida se contrastó con su replay del servidor: toda jugada confirmada figura una sola vez.
- Las jugadas se repartieron entre las réplicas: 261 y 257 en las dos nuevas y 240 en la anterior, con partidas cuyos jugadores quedaban en réplicas distintas.
- La métrica del servidor ashenfall.shot.latency (desde que llega la jugada hasta repartirla), leída en Application Insights, dio media de 15,6 ms en una primera carga corta (91 jugadas, máximo de 61 ms) y de 17,7 y 17,9 ms en las dos réplicas nuevas; el máximo en estas últimas fue de 203 ms en una jugada.
- De extremo a extremo con internet de por medio, una carga corta dio p50 de 141 ms y p95 de 216 ms: el p95 supera 200 ms por el viaje desde Costa Rica, no por el servidor.
- No se midió el punto de quiebre dentro de Azure; el de una réplica está medido en local.

8. Azure, infraestructura como código y CI/CD

8.1 Recursos

Recurso	Configuración
Container Apps (servidor)	Revisiones múltiples con división de tráfico; prod con 2 a 3 réplicas de 0,5 vCPU y 1 GiB; sondas /health/live y /health/ready
Container Apps (Redis)	0,25 vCPU y 0,5 GiB, ingress solo interno
PostgreSQL Flexible Server	Burstable B1ms, 32 GB, sin alta disponibilidad, subred delegada, acceso público desactivado, autenticación con Entra
Key Vault	jwt-secret y cadena de Application Insights, leídos con identidad administrada
Container Registry	SKU Basic, imagen del servidor etiquetada con el sha del commit
Application Insights y Log Analytics	OpenTelemetry; retención de 30 días; 3 alertas (reinicios, CPU de PostgreSQL y latencia)
Identidad administrada	Una identidad de usuario para ACR, Key Vault y PostgreSQL; ninguna clave en la configuración

Todos los recursos están en Bicep (infra/main.bicep) y un ambiente nuevo se levanta con una sola ejecución del pipeline. Los requisitos previos (identidad de GitHub con credenciales federadas, roles en la suscripción y registro de proveedores) se hacen una vez desde el portal y están en docs/azure-setup.md.

8.2 Pipeline

- **CI** (GitHub Actions) en cada pull request: build, linter, pruebas, validación de contratos, CodeQL, gitleaks, escaneo de la imagen con Trivy; Dependabot para dependencias.
- **Despliegue** con inicio de sesión OIDC sin secretos: crea el grupo de recursos, la infraestructura base, construye y publica la imagen en ACR, crea jwt-secret en Key Vault si no existe y despliega la aplicación.
- **Publicaciones sin cortar partidas:** la revisión nueva nace al 0 % del tráfico, se prueba por su propia URL (smoke test) y solo entonces recibe el 100 %; la revisión anterior queda activa para el rollback y las más viejas se desactivan, con 120 s de gracia para cerrar conexiones de forma ordenada.
- **Prod con aprobación manual:** el environment prod exige un revisor; el trabajo espera su aprobación antes de empezar.
- **Rollback:** scripts/rollback.sh pasa el 100 % del tráfico a la revisión anterior. Se probó en producción el 7 de noviembre de 2026 y tardó **22 segundos**, con /health/ready en 200 durante el cambio (meta: menos de 5 minutos).

8.3 Límite de la suscripción y decisión

Azure for Students permite un solo entorno de Container Apps por suscripción (el despliegue falla con el error MaxNumberOfGlobalEnvironmentsInSubExceeded; se comprobó incluso con dev en otra región). Por eso dev y prod no pueden existir a la vez. Dev se desplegó desde cero con el pipeline, se probó y se borró para dejar sitio a prod; el despliegue automático a dev (AUTO_DEPLOY) queda apagado. El Bicep y el pipeline de dev siguen en el repositorio y funcionan en una suscripción sin ese límite, con dev automático tras cada CI verde y prod con aprobación manual. Con dev borrado, el pipeline admite lanzar solo prod.

9. Observabilidad

Elemento	Detalle
Métricas propias (OpenTelemetry → Application Insights)	ashenfall.matches.active (partidas activas, en todas las réplicas), ashenfall.connections, ashenfall.shots (jugadas; la tasa por segundo se calcula al consultar) y ashenfall.shot.latency (latencia por jugada).
Trazas	Un span por comando (match.shoot, match.turn_timeout, match.player_disconnected…) con ashenfall.match_id y ashenfall.player_id; se comprobó en Application Insights.
Logs estructurados	Con el id de la partida y del jugador y el id de traza para cruzarlos; sin cuerpos ni cabeceras de autorización.
Sondas	/health/live (el proceso responde) y /health/ready (PostgreSQL y Redis disponibles).
Alertas	Tres alertas activas en Azure: reinicios del contenedor, CPU de PostgreSQL y latencia.

10. Pruebas y calidad

Hay 294 pruebas en 24 archivos (resultado del 7 de noviembre de 2026), que CI ejecuta en cada pull request junto con el linter y la revisión de tipos. Las pruebas de integración usan PostgreSQL y Redis reales.

Tipo de prueba	Qué cubre	Archivos
Dominio y física	Reglas, turnos, daño, terreno y obstáculos sorteados con su solución, vista por jugador, progresión y bot	match, physics, view, progression, bot
API	Usuarios, salas, historial, tienda, administración y contratos sincronizados con el código	auth, rooms, history, shop, admin, contracts
WebSocket y concurrencia	Orden, idempotencia, jugadas simultáneas, varias instancias, reconexión, abandono y sync	gateway, event-store, matchmaking
Resiliencia	Reinicio de una réplica, log dañado en una partida sin afectar a las demás, limpieza	resilience, pool, health
Seguridad	Autorización por partida y por rol, límites de tasa, cabeceras, logs sin secretos, no filtración al rival	hardening, rate-limit, password-hasher
Compras y recompensas	Compras concurrentes y recompensa duplicada	shop, event-store

Tipo de prueba	Qué cubre	Archivos
Observabilidad y estáticos	Métricas, trazas y servicio de ficheros precomprimidos	telemetry, static, config

Además de las pruebas automáticas hay dos bots reproducibles: `apps/bot/src/load.ts` (carga y punto de quiebre) y `apps/bot/src/redeploy.ts` (partidas abiertas con clientes que se reconectan y reenvían con el mismo actionId, contrastadas con el replay). Probé este último reiniciando un servidor local a media partida: 6 reconexiones y 98 de 98 jugadas confirmadas y coincidentes con el replay.

11. Cómo jugar

Se necesita solo un navegador; en el móvil se juega en horizontal y, si se sostiene en vertical, el juego pide girar el dispositivo. Para crear una cuenta basta un correo y una contraseña de al menos 10 caracteres.



Figura 4. Pantalla de acceso: crear cuenta o entrar.



Figura 5. El pueblo de Ashenfall: barra inferior con Personajes, Jugar y Perfil, el Arsenal, el Historial y los ajustes.

Elegir cómo pelear. El botón Jugar ofrece buscar partida (cola rápida contra otra persona, con espera máxima de un minuto), modo historia (un monstruo del servidor) y partida privada (crear sala y compartir el código, o unirse con el código de un amigo).

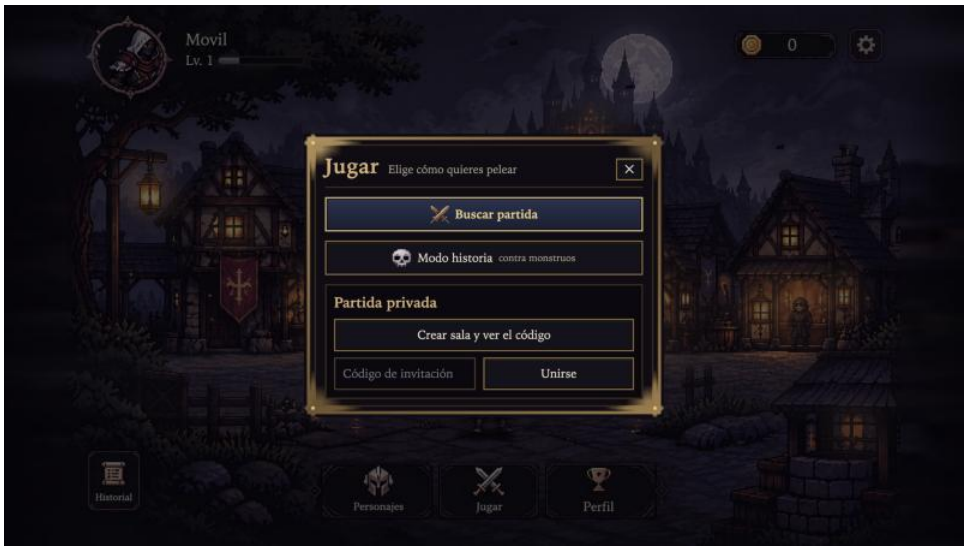


Figura 6. Menú Jugar.

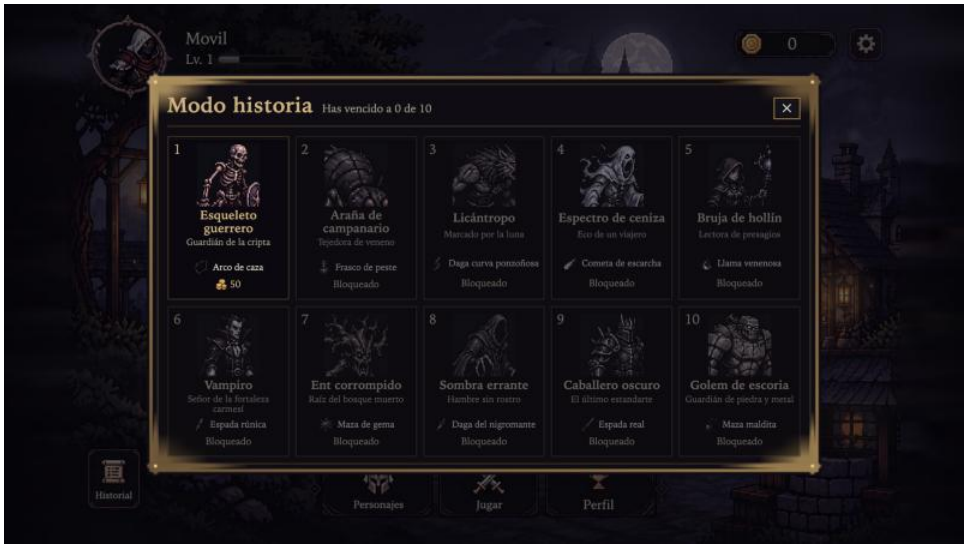


Figura 7. Modo historia: diez monstruos que se desbloquean en orden.

El duelo. Arriba se ven la vida de cada uno, de quién es el turno, los segundos restantes y la ronda; abajo, el arma y la potencia. Para disparar se mantiene pulsado el botón del ratón (o el dedo) y se arrastra hacia atrás, lejos del rival: cuanto más lejos, más fuerza, y subir o bajar el cursor cambia la altura. Una guía de puntos muestra la trayectoria con la potencia y el ángulo en porcentaje. Al soltar se dispara; Esc o el clic derecho cancelan el apuntado.



Figura 8. Arena con la guía de puntería, el turno, la vida y la potencia.

- Nadie se mueve a voluntad: solo se retrocede al recibir un impacto, según el daño. No hay viento y los obstáculos frenan los proyectiles.
- Si se agota el tiempo se pierde el turno; dos turnos agotados seguidos cuentan como abandono.
- Si alguien se desconecta tiene un minuto para volver y recibe el estado actual de la partida.

- Ganar da 100 Marcas de Ceniza; perder o empatar no da nada. Cada partida suma 10 de experiencia y 30 más si se gana; cada nivel pide 80, hasta el nivel 20.



Figura 9. Perfil: personaje, nivel, experiencia, victorias, derrotas y empates.

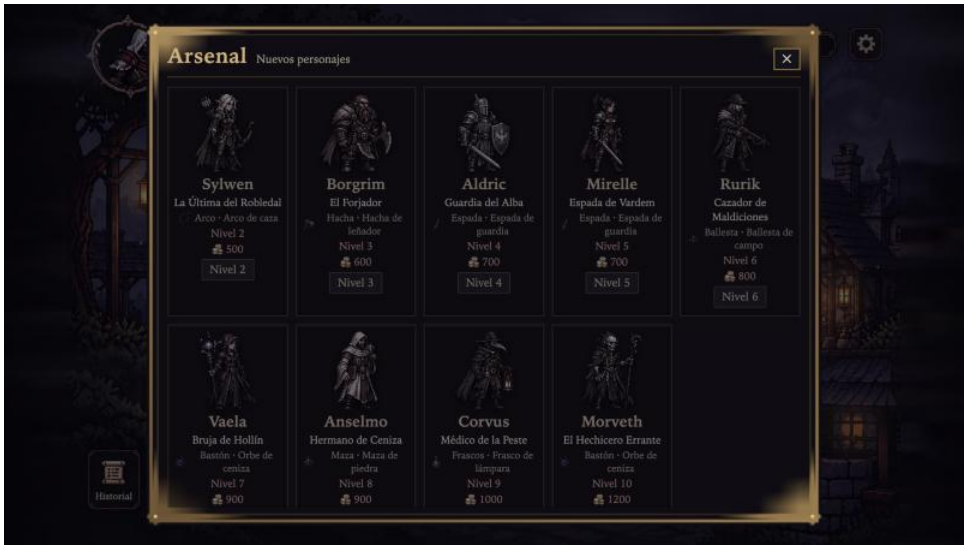


Figura 10. Arsenal: los nueve personajes que se compran con nivel y Marcas de Ceniza.

Se empieza con Edrik, el Errante. El Arsenal vende a Sylwen (nivel 2, 500), Borgrim (3, 600), Aldric (4, 700), Mirelle (5, 700), Rurik (6, 800), Vaela (7, 900), Anselmo (8, 900), Corvus (9, 1 000) y Morveth (10, 1 200); cada uno usa siempre su propia arma. En Ajustes se controla el volumen de la música y de los efectos y está el correo para reportar problemas.



Figura 11. Ajustes: volumen, correo de contacto y cierre de sesión.

12. Instalación y uso

12.1 Jugar en línea

Abrir https://ashenfall-server-prod.icytree-9cfd11e3.canadacentral.azurecontainerapps.io en un navegador y crear una cuenta. La documentación de la API está en https://ashenfall-server-prod.icytree-9cfd11e3.canadacentral.azurecontainerapps.io/docs. Para probar un duelo entre humanos con una sola persona, abrir el juego en dos navegadores (o en una ventana normal y otra privada) con una cuenta distinta en cada uno.

12.2 Correr en local (menos de 10 minutos)

Solo se necesita Docker Desktop abierto; no hace falta Node ni configurar secretos.

```
git clone https://github.com/kevinmirandaa/ashenfall-game.git
cd ashenfall-game
docker compose up --build
```

Levanta PostgreSQL, Redis y el servidor; las migraciones se aplican al arrancar. El juego queda en http://localhost:8080, la API en http://localhost:8080/docs y las sondas en /health/live y /health/ready. La imagen del servidor se construye sin caché en alrededor de un minuto en un Mac con Docker Desktop (medido el 7 de noviembre; depende de la red).

12.3 Correr las pruebas

```
npm ci
docker compose up -d --wait postgres redis
npm test
```

Otros comandos: `npm run lint`, `npm run typecheck`, `npm run build` y `npm run test:coverage`. La prueba de carga con el bot está explicada en docs/load-report.md.

12.4 Administración

Ninguna ruta de la API concede el rol de administrador. Para dárselo a una cuenta ya registrada:

```
docker compose exec server node apps/server/dist/db/grant-admin-cli.js
correo@ejemplo.com
```

13. Costo estimado en Azure

Es una estimación con precios de lista consultados el 5 de noviembre de 2026 (región northeurope, USD), no una factura; los precios cambian por región. Supuestos: mes de 30 días, sin descontar cuotas gratuitas y sin tráfico de salida. Detalle y supuestos en docs/azure-cost.md.

Recurso	Configuración	USD por mes
PostgreSQL B1ms (cómputo)	Burstable, sin alta disponibilidad	13,14
Container Registry Basic	SKU mínimo	5,00
Redis (Container App)	0,25 vCPU y 0,5 GiB, 1 réplica fija	5,8 a 19,4
Servidor de prod	0,5 vCPU y 1 GiB, 2 réplicas mínimas	23,4 a 77,8 (11,7 a 38,9 por réplica)
Log Analytics, Key Vault y alertas	Retención de 30 días, tope de 1 GB al día	desde 0; peor caso 69 en logs
Total aproximado de prod	Sin almacenamiento de PostgreSQL ni logs	**~47 a ~115**

Azure for Students suele dar 100 USD de crédito (confirmarlo en el portal), así que el costo es ajustado: se redujo con contenedores de tamaño mínimo, sin copiar métricas de plataforma a Log Analytics y sin registrar las sondas. Dev no se mantiene encendido: se borra tras probar. Pasado el periodo de revisión se recomienda borrar el grupo de recursos de producción para no gastar crédito.

14. Planeación y proceso

El trabajo se organizó con el flujo que pide el curso: un ticket de Jira por tarea (proyecto SCRUM), una rama por ticket, commits con la clave del ticket, un pull request por ticket y los estados de Jira movidos a medida que avanzaba cada uno. Hay diez épicas (SCRUM-5 a SCRUM-14) y más de 50 pull requests fusionados.

Épica	Contenido
SCRUM-5 Fundación del repo y flujo de trabajo	Repositorio, ramas, convenciones, CI inicial

Épica	Contenido
SCRUM-6 Dominio puro del duelo	Reglas, turnos, física, pruebas del dominio
SCRUM-7 API REST y usuarios	Registro, sesiones, salas, historial, administración
SCRUM-8 Tiempo real por WebSocket	Protocolo, ticket, heartbeat, validación de mensajes
SCRUM-9 Persistencia y multi-réplica	Log de eventos, Redis, orden e idempotencia entre réplicas
SCRUM-10 Seguridad	Argon2id, JWT, límites de tasa, modelo de amenazas
SCRUM-11 Cliente del juego	Cliente de prueba con Phaser, pueblo, arena, tienda
SCRUM-12 Azure, IaC y CI/CD	Bicep, GitHub Actions, identidad, publicaciones y rollback
SCRUM-13 Observabilidad	OpenTelemetry, métricas propias, sondas, alertas
SCRUM-14 Rendimiento y entrega	Prueba de carga, optimizaciones, documentación y etiqueta v1.0.0

En la etapa final se hicieron, cada una con su ticket y su pull request: personajes comprables con nivel y dinero (SCRUM-62 a 66), terreno y obstáculos aleatorios por partida (67), correcciones de las pruebas del usuario (68), QA y limpieza (69), optimización medida (70), auditoría contra el enunciado (72), despliegue de producción (73), README con la historia y cómo jugar (74), publicación solo a prod y rollback probado (75 y 76), verificación en Azure y soporte móvil (77 a 79), guía de juego y arquitectura (80) y esta documentación (81). Las decisiones de arquitectura y de juego están en docs/DECISIONS.md.

15. Cumplimiento del enunciado

La matriz completa, con 94 requisitos, su evidencia (código, prueba, documento, workflow o Bicep) y su estado, está en docs/cumplimiento-enunciado.md. Resultado: **92 requisitos cumplidos, 2 cumplidos parcialmente y ninguno incumplido**. Resumen por criterio de la rúbrica:

Criterio (peso)	Evidencia principal	Estado
Arquitectura y calidad de construcción (20 %)	Dominio puro aislado y verificado por ESLint; capas; TypeScript estricto sin any; arquitectura documentada con diagramas	Cumple
Sesiones y concurrencia (20 %)	Orden por partida, idempotencia, reconexión, abandono, plazos en PostgreSQL, varias réplicas probadas en Azure, publicación con partidas abiertas sin pérdidas	Cumple
Seguridad (20 %)	Autenticación y autorización por partida en HTTP y WebSocket, validación de cada mensaje, límites de tasa, sin filtración al rival, secretos en Key Vault, modelo de amenazas, sin secretos en el repositorio	Cumple
Rendimiento y optimización (15 %)	Prueba de carga reproducible; p95 de 15 ms o menos hasta 800 partidas en local; punto de quiebre medido (~1 800 jugadas/s por réplica); dos optimizaciones con cifras; mediciones en Azure	Cumple

Criterio (peso)	Evidencia principal	Estado
Despliegue en Azure, IaC y CI/CD (10 %)	Bicep, pipeline con OIDC, identidad administrada, prod con aprobación, publicaciones sin cortes, rollback de 22 s, ambiente levantado desde cero por el pipeline	Cumple (dev automático parcial por el límite de la suscripción)
Estándares, contratos y pruebas (10 %)	OpenAPI 3.1 y AsyncAPI 3.0 sincronizados con el código, Problem Details, 294 pruebas unitarias, de integración, concurrencia, seguridad y resiliencia en verde en CI	Cumple
Documentación y observabilidad (5 %)	Este documento y la carpeta docs/; logs y trazas con id de partida y jugador; cuatro métricas propias; sondas; tres alertas	Cumple

16. Limitaciones y trabajo futuro

- **Dev automático:** Azure for Students permite un solo entorno de Container Apps por suscripción, así que dev y prod no pueden coexistir y el despliegue automático a dev quedó apagado. Es el único requisito que cumple parcialmente por una restricción externa.
- **Información oculta:** es opcional en el enunciado; la posición del rival y la trayectoria son públicas por diseño. Sí hay vista por jugador y prueba de no filtración.
- **Latencia en Azure:** la carga se hizo desde internet y en escalones pequeños (hasta 8 partidas). La latencia medida en el servidor es baja (media de unos 17 ms), pero el punto de quiebre solo está medido en local y habría que repetirlo dentro de Azure con carga alta.
- **Navegadores:** el cliente se probó en Chromium, en un navegador de escritorio y en emulación móvil horizontal. Safari y Firefox no se pudieron probar con navegadores reales; el código se compila para Safari 14 o superior, Firefox 78 y Chrome 87.
- **Cliente:** el cliente es de prueba; no tiene nota. El móvil solo se juega en horizontal.
- **Mejoras posibles:** alertas por correo (el parámetro existe y falta pasarlo en el pipeline), un rastreo de dependencias con las actualizaciones mayores de Dependabot (TypeScript 7, ESLint 10 y Vitest 5 están pendientes de revisar) y medir el punto de quiebre dentro de Azure.

17. Conclusión

ASHENFALL cumple lo que pide el proyecto: un servidor autoritativo, seguro y observable, con muchas partidas simultáneas e independientes, orden e idempotencia de las jugadas, reconexión y abandono, varias réplicas coordinadas por PostgreSQL y Redis, y publicaciones sin cortar partidas, desplegado en Azure con infraestructura como código, identidad administrada y un pipeline con aprobación manual a producción. Las cifras de rendimiento y de resiliencia salen de ejecuciones reales y reproducibles con los bots incluidos en el repositorio, y donde algo no se pudo comprobar o la suscripción lo impide se dice en este documento.

El juego está disponible en
https://ashenfall-server-prod.icytree-9cfd11e3.canadacentral.azurecontainerapps.io durante el

periodo de revisión, y el código, la infraestructura, los pipelines, los contratos, las pruebas, los bots y la documentación están en https://github.com/kevinmirandaa/ashenfall-game, en la versión etiquetada v1.0.0.